

Monitoring Human Mobility with Smartphones

CS3244 Machine Learning – Group 32 Final Project Report

Elysia Selena Alston, Jiuyan Lu, Nabil Rakaiza Abror, Tiara Mirchandani, Wang Kaitian, Zakariyyaa Chachia

1. Introduction

Human activity recognition is an important machine learning problem with applications in areas such as activity monitoring, health tracking, fitness analysis, etc. Accurately identifying activities like walking, sitting, or posture transitions allows systems to better understand human behaviour and provide timely feedback or interventions when needed. There has been a growing interest in building reliable and scalable activity recognition systems.

Many existing solutions rely on camera-based setups or computer vision techniques to detect and classify movement. However, these approaches are often limited to specific locations, making it difficult to scale beyond controlled movements. They also introduce privacy concerns that may make users uncomfortable with their visual data being captured and used to train models. These limitations motivate the need for alternative approaches that are both scalable and preserve privacy.

Our project aims to provide a low cost alternative by classifying human activities using smartphone sensor data. Modern smartphones are equipped with inertial sensors that can continuously capture motion signals, making them a practical platform for activity recognition without requiring visual information. In this project, we aim to classify twelve activity labels using smartphone sensor signals, covering static postures, dynamic movements, and short postural transitions. This classification type has a wide range of applications in activity monitoring, where a smartphone, such as the one used to gather the data, could be used to monitor the user's activity throughout the day and suggest appropriate times for movement or exercise in a safe manner. Smartphone users would ultimately benefit as they get motivated by reminders sent to their phones based on their activity levels. For example, if a user remains seated for a long period of time, the system could remind them to move around.

To address this task, we explore and compare multiple machine learning models with varying levels of complexity, including a logistic regression baseline, a kernel-based Support Vector Machine, and deep learning models such as a 1D-Convolutional Neural Network and a CNN-LSTM architecture. Beyond overall accuracy, we focus on understanding model behaviour under class imbalance, particularly for rare postural transitions. Using evaluation metrics such as macro F1-score, confusion matrices, and ROC-AUC, we aim to systematically analyse model performance and identify the strengths and limitations of different approaches for smartphone-based human activity recognition.

2. Dataset & Problem Analysis

2.1 Dataset Description

The dataset used in this project is the Recognition of Human Activities Using Smartphones dataset, which contains IMU data from a smartphone with 30 users completing different actions. Each participant performed a predefined set of daily activities while carrying a smartphone that records the accelerometer and gyroscope signals. These raw signals are then processed into a set of engineered features, resulting in a total of 561 features per instance.

The dataset contains over 10,000 samples across 12 activity classes including static postures (SITTING, STANDING, LYING), dynamic activities (WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS), and postural transitions (e.g. SIT_TO_STAND, STAND_TO_LIE). The feature set includes both time domain and frequency domain statistics to capture different aspects of motion and orientation during each activity.

All features are pre-normalised to the range $[-1, 1]$, which simplifies preprocessing and allows models with different optimisation characteristics to be trained consistently. This allows us to use the dataset for comparing traditional machine learning models and neural network-based approaches under a shared experimental setup.

2.2 Class Imbalance and Its Implications

After exploring, we found that this dataset has a severe imbalance between activity classes. The six static and dynamic activities each contain hundreds to over a thousand samples, while the other six postural transition classes have significantly fewer samples, some even lower than 25. This imbalance introduces a major challenge for classification as models trained to optimise overall accuracy are likely to prioritise majority classes

while largely ignoring rare transitions. Some postural transitions also share similar patterns (e.g. SIT_TO_STAND and STAND_TO_SIT), making them inherently difficult to distinguish. When combined with limited training data, this increases the likelihood of poor classification for minority classes.

Hence, naive evaluation using accuracy alone can be misleading. A model that performs well on majority classes may still appear to achieve high accuracy while failing almost entirely on transition activities. This motivated the use of evaluation metrics that are class sensitive and balancing strategies in later stages of the project.

3. Related Works

3.1 Traditional Machine Learning and the Transition to Non-Linearity

Human Activity Recognition (HAR) utilizing smartphone inertial sensors has been extensively studied, with researchers employing a wide spectrum of methodologies to address this complex classification problem. Early baselines often utilized linear classifiers, such as Logistic Regression (Dupare, 2023; Rabbi, 2021). While computationally lightweight and easy to interpret, linear models typically struggle to capture the complex, non-linear decision boundaries inherent in human kinetic data, often resulting in suboptimal performance.

Interestingly, Logistic Regression has been reported to achieve accuracies of around 96% (Dupare, 2023; Rabbi, 2021). This strong performance may be attributed not to inherent linearity in the raw data, but to the effectiveness of the engineered feature set, which likely transforms the original signals into a space that is closer to linearly separable.

To further address non-linearities, subsequent studies established Support Vector Machines (SVMs) as a strong benchmark. Foundational work on the UCI HAR dataset demonstrated that SVMs with non-linear kernels can also achieve accuracies around 96% (Anguita et al., 2013). However, this performance heavily depends on a carefully hand-crafted 561-dimensional feature vector derived from both time and frequency domains. Because feeding 561 distinct features into simpler linear models or SVMs often leads to high computational overhead and potential overfitting, the literature frequently incorporates dimensionality reduction techniques, such as Principal Component Analysis (PCA). By projecting the data into a lower-dimensional space, researchers can retain the most critical variance while optimizing the models for faster training. While effective, this approach requires substantial domain expertise and preprocessing effort, limiting its scalability to new sensor modalities or unseen activities.

3.2 The Deep Learning Paradigm and Automated Feature Extraction

To circumvent the limitations of manual feature engineering, the research paradigm shifted towards deep learning techniques capable of automated feature extraction directly from raw data. Researchers have widely adopted 1D Convolutional Neural Networks (1D-CNNs) for this purpose (Muralidharan et al., 2021). Drawing inspiration from their success in image processing, 1D-CNNs effectively extract spatial and structural features from raw accelerometer and gyroscope windows. By processing the raw data directly, CNNs can uncover hidden, meaningful relationships that might otherwise be lost during manual feature transformation. However, standalone CNN architectures are inherently limited by their inability to model the temporal, sequential nature of continuous human movement.

3.3 Spatiotemporal Modeling via Hybrid Architectures

To bridge this gap and introduce temporal context, hybrid architectures combining CNNs with Long Short-Term Memory (LSTM) networks have emerged as a dominant approach in the literature. In these models, the CNN layers function as automated spatial feature extractors, while the LSTM layers process the resulting feature sequences to capture temporal dependencies over time (Sharma et al., 2025). This spatiotemporal modeling is particularly critical for distinguishing complex or dynamic activities. While contemporary literature has begun exploring Transformer-based architectures to mitigate issues such as the vanishing gradient problem inherent in recurrent networks (Liang, 2023), CNN-LSTMs remain a recognized gold standard, offering a highly effective balance between sequential modeling power and architectural stability.

3.4 Data Imbalance and Robust Evaluation Metrics

A recurring challenge highlighted in HAR literature is the inherent class imbalance within standard datasets. Transient movements (e.g., transitioning from standing to sitting) occur far less frequently than stationary or repetitive activities like walking. Consequently, models trained on imbalanced data are prone to majority-class bias, prioritizing the dominant classes while neglecting the minority transitions. To mitigate this,

researchers propose techniques such as the Synthetic Minority Over-sampling Technique (SMOTE) or the implementation of class-weighted loss functions to ensure balanced learning. Demonstrating the efficacy of this approach, Arafa (2026) highlighted that a SMOTE-enhanced CNN-Bi-LSTM architecture outperforms several state-of-the-art methods.

Furthermore, relying solely on overall accuracy can be misleading, particularly in imbalanced classification scenarios. Therefore, additional evaluation metrics such as precision, recall, and F1-score are also employed. In particular, the Macro F1-score is widely adopted as a more robust measure, as it evaluates model performance uniformly across both majority and minority classes.

3.5 Research Motivation and Deployment Considerations

While existing research extensively details the efficacy of these models in isolation, there is significant value in a direct, comparative benchmark on the same dataset. Furthermore, because HAR models are ultimately designed for mobile and wearable devices, evaluating the computational complexity of these algorithms is vital. By comparatively analyzing Logistic Regression, SVM, 1D-CNN, and a hybrid CNN-LSTM, this report aims to evaluate not only predictive accuracy and temporal sequence handling but also the operational trade-offs between manual feature engineering, raw signal processing, and the computational efficiency required for potential edge deployment.

4. Implementation

4.1 Simple Regression

The Logistic Regression model is implemented as a baseline classifier for the 12-class human activity recognition task. The objective of this model is to provide a simple and interpretable benchmark before introducing more complex non-linear models. The input consists of the pre-engineered 561-dimensional feature vectors derived from accelerometer and gyroscope signals. Prior to model training, a structured preprocessing pipeline is applied to ensure proper data handling and to prevent data leakage.

Preprocessing begins with StandardScaler to normalize all features to zero mean and unit variance. This is necessary because Logistic Regression is sensitive to feature scaling, and unscaled features may lead to unstable optimization. A SelectKBest method with ANOVA F-statistic (f_{classif}) is used to select the top k most informative features. This step reduces noise and mitigates the curse of dimensionality. The value of k is treated as a hyperparameter and tuned during model selection. The classifier is a multi-class Logistic Regression model using a one-vs-rest (OvR) scheme. Class imbalance is addressed by setting `class_weight='balanced'`, which adjusts the loss function to give higher importance to minority classes. The optimization is performed using the 'saga' solver, which is more suitable for high-dimensional data and large-scale problems. The maximum number of iterations is increased to ensure convergence.

To ensure robust evaluation and avoid data leakage, all preprocessing steps and the classifier are integrated into a unified pipeline using `sklearn.pipeline.Pipeline`. Hyperparameter tuning is performed using `GridSearchCV` with 5-fold cross-validation. The search space includes a number of selected features: $k \in \{100, 200, 300\}$ and regularization strength: $C \in \{0.1, 1, 10\}$. The model is optimized using macro F1-score as the evaluation metric during cross-validation. This choice is motivated by the class imbalance in the dataset, where accuracy alone may be dominated by the majority classes. The best-performing configuration is then evaluated on the held-out test set.

4.2 Kernel Support Vector Machine

4.2.1 SVM-A: Feature Space and Kernel Ablation

The progress report expanded upon a working RBF Kernel SVM pipeline (StandardScaler $\rightarrow$ PCA at 95% variance $\rightarrow$ SVM with balanced class weights), tuned via 5-fold grid search, that achieved 93% accuracy and a weighted F1-score of 0.93. For the final report, we set out to justify each of those design choices and to test the two suggestions we had made for improvement: engineering custom Z-axis features to distinguish SITTING from STANDING, and comparing against temporal models.

The main issue with the progress-report setup was that we could not tell, from one tuned pipeline, which components were actually helping. A four-axis ablation study was therefore done, keeping the pipeline otherwise fixed: (i) kernel choice (linear, polynomial, RBF), (ii) degree of PCA compression (raw 561 features, PCA-99%, PCA-95%, PCA-90%), (iii) class weighting on vs off, and (iv) a handcrafted feature subset restricted to gravity, Z-axis, and orientation-angle features selected from the UCI feature names. All configurations were scored on macro F1 on the held-out test set. Macro F1 was chosen over accuracy because of the severe class

imbalance described in Section 2.2: accuracy is dominated by the six large classes and can remain high even when the six postural-transition classes are almost entirely misclassified. Macro F1 weights every class equally and is therefore the metric that reflects actual performance on the minority classes.

After the ablation, we re-ran grid search on the best configuration over a wider range ($C \in \{1, 10, 100\}$, $\gamma \in \{\text{scale}, 0.01, 0.05\}$) and verified stability by repeating 5-fold cross-validation across five random seeds.

4.2.2 SVM-B: Oversampling and Hierarchical Decomposition

The progress report established a working kernel SVM pipeline (VarianceThreshold $\rightarrow$ SelectKBest at $k=200 \rightarrow$ StandardScaler $\rightarrow$ RBF SVM with balanced class weights) that reached 93% test accuracy and macro F1 of 0.82. The confusion matrix showed all six transition classes being confused with each other despite balanced class weights. Since the ablation in Section 5.2.1 concluded that class reweighting alone is ineffective on this dataset, SVM-B tests data-level oversampling and then pushes further into architectural decomposition.

Two versions were built. Version 1 adds SMOTE inside the existing pipeline, placed within an imblearn.Pipeline so that oversampling only runs on training folds during cross-validation and never leaks into validation folds. Each transition class was oversampled to 400 synthetic instances to match the lower end of the static class counts. Grid search over $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{\text{scale}, \text{auto}, 0.001, 0.01\}$ selected $C=100$, $\gamma=\text{auto}$.

Version 2 replaces the flat classifier with a hierarchical two-stage model, motivated by the observation that the six static/dynamic classes and six postural transitions are fundamentally different sub-problems. The static classes form well-separated clusters in feature space, while the transitions overlap heavily. A single SVM must compromise between both; version 2 eliminates this by splitting into three specialists. Stage 1 is a binary router classifying each sample as static or transition, with an asymmetric confidence threshold of 0.75: a sample is routed to the static specialist only if the router exceeds 75% confidence, because misrouting a transition to the static branch is guaranteed wrong (that specialist has never seen transition labels). Stage 2a is an RBF SVM trained only on the static/dynamic classes with no oversampling. Stage 2b is an RBF SVM for transitions only, with two changes: SMOTE was replaced by ADASYN, which concentrates synthetic samples near decision boundaries rather than interpolating uniformly, better matching the overlapping transition geometry; and SelectKBest was switched from the ANOVA F-statistic to mutual information to capture non-linear feature-class dependencies. Hyperparameter tuning for the transition specialist used macro F1 as the objective (grid over $k \in \{100, 200, 300, 400\}$, $C \in \{10, 100, 1000\}$, $\gamma \in \{\text{scale}, 0.001, 0.01\}$). After tuning, the full model was refit on train+val combined to maximise real training samples for the transition branch.

4.3 One-Dimensional Convolutional Neural Network (1D-CNN)

Instead of feeding the 561-dimensional feature vector directly into the network, we treated each sample as a one dimensional signal of length 561 with a single channel, reshaping the input from (7767, 561) to (7767, 561, 1). Conv1D layers slide a small kernel across the feature axis and can pick up local patterns between neighboring features. A plain dense layer would treat all 561 features as independent and miss the hidden structure entirely. Furthermore, since Keras expects 0 indexed classes, labels were shifted from 1-12 to 0-11. Sparse_categorical-crossentropy was also used as the loss so integer labels could be passed directly without needing one-hot encoding.

The architecture itself consists of two Conv1D blocks followed by GlobalAveragePooling1D, then a small dense head. The first convolutional layer uses 64 filters with kernel size 3 and same-padding, with the second layer doubling to 128 filters with the same kernel size. The filters increase in the second layer to provide more capacity to combine lower level patterns detected in the first layer into higher level representations. Kernel size 3 was chosen because it was small enough to capture local feature interactions without being so wide that it just averages over everything. Same-padding keeps the sequence length at 561 throughout both conv layers, so no information is lost at the edges. Each conv layer is followed by BatchNormalization, which functions to normalise the activations within each batch during training. Without normalisation, some features would dominate the gradient updates and others would barely contribute.

After the two conv blocks, GlobalAveragePooling1D collapses all 561 positions into a single vector by averaging across the feature axis. The alternative would have been Flatten, but that would produce a $561 \times 128 = 71808$ dimensional vector going into the dense layers, which would increase parameter count and increase risk of overfitting given the relatively small training set. GlobalAveragePooling1D brings this down to just 128 values, acting as a built-in regulariser while preserving key information extracted by the conv layers. The pooled

vector then goes through a Dense(64) layer with ReLU, a dropout(0.3) layer, and a final Dense(12, softmax) output. The total model comes to 34764 parameters, which is intentionally lightweight, as we aim for a model that trains quickly and does not need a large dataset to converge.

For training, Adam was chosen as the optimiser at its default training rate because it adapts the learning rate per parameter based on past gradients. It can automatically handle sparse or infrequent gradient updates, making it suitable for our 12-class problem with imbalanced classes and relatively few samples in some classes. Training was run for up to 200 epochs with batch size 64 and a 10% validation split. An EarlyStopping callback monitored each validation accuracy with patience 10 and restore_best_weights = True, so training would stop once the model stopped improving and automatically revert to its best checkpoint rather than continuing to overfit.

To mitigate the class imbalance, we initially tested using class weights but it degraded macro F1, so the final model relies on Dropout(0.3), BatchNormalization, and GlobalAveragePooling1D for implicit regularisation. All random seeds were fixed at 3244 for reproducibility.

4.4 Convolutional Neural Network – Long Short-Term Memory (CNN-LSTM)

To address this classification problem, a Convolutional Neural Network-Long Short-Term Memory (CNN-LSTM) framework is utilized. The proposed methodology evaluates the model under two distinct experimental conditions: one utilizing raw temporal sensor sequences and the other utilizing pre-computed engineered features. Furthermore, to mitigate the inherent class imbalance within the dataset, two counter-measures are systematically investigated: the application of class-weighted loss functions and the Synthetic Minority Over-sampling Technique (SMOTE).

For the engineered feature dataset, the training process is configured for a maximum of 100 epochs utilizing a batch size of 64, a learning rate of 0.001, and a 20% validation split. To prevent overfitting, dropout layers are applied with a rate of 0.5, and an early stopping mechanism is implemented with a patience of 10 epochs monitoring the validation loss. The architecture consists of a CNN block featuring two 1D Convolutional layers (64 filters each) paired with Batch Normalization, followed by 1D Max Pooling. The sequential features are then processed by two consecutive LSTM layers (64 units each). The network concludes with two fully connected Dense layers, ultimately mapping to the 12 target classes, resulting in a model comprising 84,108 total parameters.

For the raw temporal data, sequences are processed using a sliding window approach with a window size of 128 timesteps and a 50% overlap. To ensure robust evaluation, the dataset is partitioned by subject, utilizing users 1 through 26 for training and users 27 through 30 for testing. Training hyperparameters mirror the engineered setup, including a batch size of 64, a learning rate of 0.001 and a 100-epoch limit. The architecture designed for the raw inputs begins with a deeper CNN block comprising three 1D Convolutional layers that progressively expand the feature dimensions, interleaved with Batch Normalization, ReLU activations, 1D Max Pooling, and Dropout. The extracted feature maps are then permuted and fed into a two-layer LSTM block (128 hidden units). Finally, the temporal output from the final timestep is extracted and passed through a classifier head, which consists of a Dropout layer, a fully connected layer reducing the dimensions from 128 to 64 (with ReLU activation), and a final linear projection to the target classes. In total, the model contains 312,012 total parameters.

5. Results & Interpretation

5.1 Simple Regression

The final Logistic Regression model achieves an overall accuracy of approximately 93% on the test set. The weighted F1-score is also 0.93, indicating strong performance on the majority classes. However, the macro F1-score is significantly lower at approximately 0.83, reflecting performance disparities across classes.

```

Accuracy: 0.9263124684688582
F1 (macro): 0.8252636924617143
Classification Report:

```

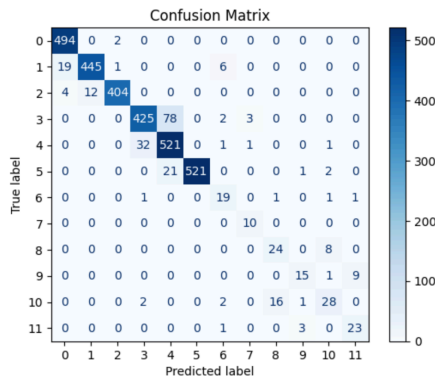
	precision	recall	f1-score	support
1	0.96	1.00	0.98	496
2	0.97	0.94	0.96	471
3	0.99	0.96	0.98	420
4	0.92	0.84	0.88	588
5	0.84	0.94	0.89	556
6	1.00	0.96	0.98	545
7	0.61	0.83	0.70	23
8	0.71	1.00	0.83	10
9	0.59	0.75	0.66	32
10	0.75	0.68	0.72	25
11	0.68	0.57	0.62	49
12	0.70	0.85	0.77	27
accuracy			0.93	3162
macro avg	0.81	0.85	0.83	3162
weighted avg	0.93	0.93	0.93	3162

The classification report reveals a clear pattern:

The six primary activity classes (e.g., WALKING, SITTING, STANDING, LAYING) achieve high F1-scores, typically above 0.90. In contrast, the six transitional classes (e.g., SIT_TO_STAND, STAND_TO_SIT) have substantially lower F1-scores, ranging from approximately 0.62 to 0.77.

This discrepancy is primarily due to two factors. First is class imbalance. The transitional classes have significantly fewer samples (as low as 10–50 instances), making them harder to learn. Second, feature overlap. Transitional

activities exhibit highly similar feature representations, leading to confusion between classes.



The confusion matrix further supports this observation. Strong diagonal dominance is observed for the primary activity classes, indicating accurate classification. However, the transitional classes show substantial off-diagonal entries, suggesting frequent misclassification among these categories. In particular, transitions such as SIT_TO_STAND and STAND_TO_SIT are often confused with each other, indicating that the model struggles to distinguish subtle differences in movement patterns.

The model achieves a high one-vs-rest ROC AUC score of approximately 0.99, indicating strong overall separability between classes. However, this metric is influenced by the well-separated majority classes and does not fully reflect the difficulty in distinguishing minority transition classes.

After introducing a proper pipeline with cross-validation, both accuracy and ROC AUC show a slight decrease compared to earlier experiments. This is expected and desirable, as the pipeline ensures that preprocessing steps such as scaling and feature selection are applied only to the training folds during cross-validation, thereby eliminating data leakage. As a result, the reported performance is a more reliable estimate of the model's true generalization ability. Earlier higher scores were likely optimistic due to implicit leakage of information from the test set.

Despite its simplicity and efficiency, the Logistic Regression model exhibits several limitations. First limitation being its linear decision boundary. The model assumes linear separability between classes, which is insufficient for capturing the complex relationships in the data. It also has a poor performance on transition classes as it struggles with minority classes that have overlapping feature distributions. Another limitation is the limited feature interaction modeling, as logistic regression does not capture higher-order interactions between features.

5.2 Kernel Support Vector Machine

5.2.1 SVM-A: Feature Space and Kernel Ablation

The ablation produced four findings that updated my understanding from the progress report.

First, PCA is a speed tradeoff, not an accuracy improvement. RBF on the raw 561 features reached macro F1 of 0.860, vs 0.860 for PCA-99%, 0.852 for PCA-95% (the progress-report setting, which compressed to 103 components), and 0.835 for PCA-90%. Compressing to 95% variance costs about one macro F1 point for a ~2x training speed-up. The progress report described PCA as "preventing overfitting," but the data shows it is useful for computation, not regularisation.

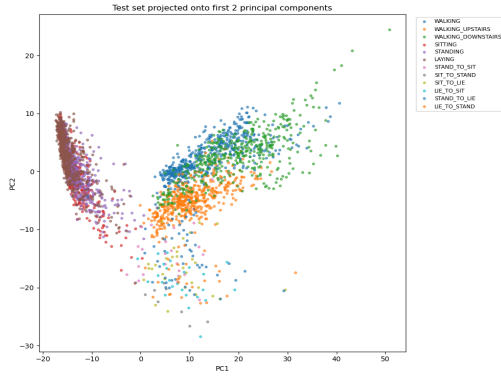
Second, balanced class weighting had no measurable effect (macro F1 = 0.8517 with or without). The soft-margin SVM's slack penalty already tolerates imbalance well enough on this dataset that reweighting adds nothing. A useful negative result showing imbalance mitigation is model-dependent.

Third, the Z-axis hypothesis from the progress report was wrong. The handcrafted gravity/Z-axis feature subset (60 features) dropped macro F1 to 0.789, seven points below the full feature set. The UCI engineered features already encode axis-specific information in combinations that manual selection broke apart.

Fourth, kernel choice mattered less than expected. At PCA-95%, RBF (0.852) only narrowly beat Linear (0.820) and Polynomial (0.804). On raw 561 features, a Linear SVM reached macro F1 of 0.848 and accuracy of 94.7%. The kernel trick is only buying about one macro F1 point, suggesting the engineered feature space is already close to linearly separable.

Table X: SVM-A ablation results on the test set, sorted by macro F1.

Configuration	Macro F1	Weighted F1	Accuracy	Train (s)
RBF, raw 561, balanced	0.860	0.942	0.942	1.5
RBF, PCA-99%, balanced	0.860	0.937	0.938	0.9
RBF, PCA-95%, balanced (progress report)	0.852	0.930	0.930	0.7
RBF, PCA-95%, no class weight	0.852	0.930	0.930	0.7
RBF, PCA-90%, balanced	0.835	0.911	0.911	0.5
Linear, PCA-95%, balanced	0.820	0.903	0.903	2.2
Polynomial, PCA-95%, balanced	0.804	0.903	0.904	0.5
RBF, handcrafted Z-axis subset	0.789	0.872	0.873	0.4



The final SVM-A configuration is RBF on raw 561 features with balanced class weights, improving on the progress-report pipeline (macro F1 0.852 $\rightarrow$ 0.860, accuracy 93.0% $\rightarrow$ 94.2%). Grid search on macro F1 (5-fold CV, $C \in \{1, 10, 100\}$, $\gamma \in \{\text{scale}, 0.01, 0.05\}$) was run on the PCA-95 pipeline for computational efficiency, giving cross-validation macro F1 of 0.894 ± 0.003 across five random seeds.

The confusion errors flagged in the progress report both persisted: 93 SITTING $\leftrightarrow$ STANDING and 49 WALKING_DOWNSTAIRS $\leftrightarrow$ WALKING_UPSTAIRS. We expected these to be borderline cases, so I checked the decision function: the gap between the top-scoring class and

the runner-up was 1.015 on average for correct predictions and 1.012 for errors; they are essentially identical. The model is not uncertain about these errors; it is confidently wrong. A 2D PCA projection of the test set confirms this: SITTING and STANDING clusters overlap substantially rather than sitting next to each other, meaning misclassified samples land inside the wrong class's region of feature space, not on the boundary between them.

Three improvements are worth trying. Since class reweighting did not help, a data-level method such as SMOTE (used in Section 5.2.2 in SVM-B and Section 5.4) would address the imbalance differently. Automated feature selection using regularised linear SVM coefficients would likely beat manual name-based selection. Most importantly, the SITTING $\leftrightarrow$ STANDING errors suggest static per-window features have hit their ceiling; whether the temporal models in Sections 5.3 and 5.4 resolve this specific confusion is the clearest test of whether temporal context is the issue.

5.2.2 SVM-B: Oversampling and Hierarchical Decomposition

Both versions hit the same macro F1 ceiling. Table Y summarises test-set performance alongside the SVM-A final configuration from 5.2.1 for comparison.

Table Y: SVM-B test-set results, with SVM-A shown for reference.

Configuration	Macro F1	Weighted F1	Accuracy	Macro AUC-ROC	MIoU
SVM-A: RBF, raw 561, balanced	0.860	0.942	0.942	0.985	0.771
SVM-B V1: flat + SMOTE	0.850	0.930	0.931	0.994	0.748
SVM-B V2: Hierarchical + ADASYN	0.850	0.930	0.929	0.996	0.758

SMOTE produced a measurable but modest improvement over the progress-report baseline (macro F1 0.82 $\rightarrow$ 0.85), confirming that data-level methods outperform reweighting on this dataset, though the three

hardest transition classes remained below 0.75 F1. Hierarchical decomposition did not meaningfully outperform the flat model: v2 improved recall on several transitions (e.g. SIT_TO_LIE 0.69 → 0.81, SIT_TO_STAND 0.90 → 1.00) but at the cost of slightly reduced precision on static classes from occasional misrouting, redistributing errors without reducing their total count.

The gap between macro AUC-ROC (0.994+) and macro F1 (0.85) is diagnostic: the SVM assigns higher scores to the correct class on most transition samples but the hard argmax is wrong because the probabilities are only marginally above the runner-up, mirroring the decision-function gap reported for SITTING↔STANDING in Section 5.2.1.

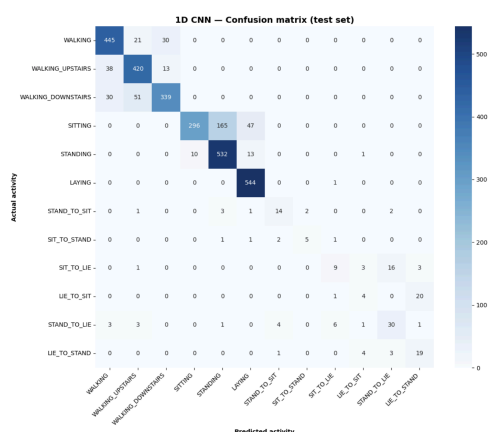
The fact that two architecturally different approaches (flat SMOTE, hierarchical ADASYN with mutual information feature selection) plateau at the same macro F1 indicates the limitation is architectural. An SVM operates on static feature vectors and cannot encode temporal ordering within a sensor window — precisely the information needed to distinguish transitions from their reverses (e.g. SIT_TO_LIE vs LIE_TO_SIT). This motivates the sequential models evaluated in Sections 5.3 and 5.4.

5.3 One-Dimensional Convolutional Neural Network (1D-CNN)

Table below summarises the overall performance of the 1D-CNN on the test set. The model achieved an accuracy of 84.03%, a macro F1-score of 0.6707, a weighted F1-score of 0.8337, a macro ROC-AUC of 0.9871, and an mIoU of 0.5421. These results show that the model learned meaningful structure from the engineered features, but that its performance was uneven across classes. In particular, the gap between weighted F1 and macro F1 suggests that the model performed substantially better on frequent classes than on minority ones, indicating that class imbalance remained a significant challenge.

Metric	Value
Accuracy	0.8403
Macro F1	0.6707
Weighted F1	0.8337
Macro ROC-AUC	0.9871
mIoU	0.5421
Parameters	34,764

Table: 1D-CNN test-set performance summary



The confusion matrix shows strong diagonal dominance for the major activity classes, indicating that the 1D-CNN correctly classified many samples in the more distinctive categories. In particular, the model performed strongly on large and well-separated classes such as LAYING, STANDING, and WALKING. However, the errors were not randomly distributed, but concentrated in a few meaningful regions, showing that the model mainly struggled with physically similar classes.

One major confusion cluster involved the posture-based classes SITTING, STANDING, and LAYING. The largest error occurred when SITTING was misclassified as STANDING, suggesting that the model had difficulty separating quasi-static postures whose differences depend more on body orientation than strong motion cues. The transition classes were the most challenging overall. Because these classes had far fewer samples than the primary activities, the model had less data to learn reliable patterns from. As a result, classes such as SIT_TO_STAND, SIT_TO_LIE, LIE_TO_SIT, and LIE_TO_STAND showed weaker and more scattered predictions. This explains why the macro F1-score was much lower than the overall accuracy.

The relatively high ROC-AUC of 0.9871 together with the lower macro F1-score of 0.6707 is also informative. It suggests that the model could often rank the correct class highly, but still failed at the final hard class assignment when similar classes competed closely. In other words, the 1D-CNN learned useful overall separability, but its decision boundaries remained imperfect for minority classes and boundary cases.

Despite 1D-CNN having greater architectural complexity, it underperforms both logistic regression and the SVM on macro F1. This is likely because the 561 engineered features are aggregate window statistics rather than raw time series signals, hence treating them as a 1D sequence does not introduce meaningful temporal structure for the Conv1D layers to exploit, and the SVM and logistic regression which treated features independently are better suited to this input format. The 1D-CNN still relies on the pre-engineered 561-feature representation rather than the original temporal sensor sequences, meaning the temporal ordering information needed to distinguish transitions from their reverses is already lost before the model sees the data. Overall, the 1D-CNN serves as a useful bridge between traditional feature-based classifiers and the more sequence-aware CNN-LSTM model in Section 5.4, reinforcing that preserving temporal information is more important than simply increasing model complexity for classifying subtle human activity transitions.

5.4 Convolutional Neural Network – Long Short-Term Memory (CNN-LSTM)

The table below presents the performance metrics of the CNN-LSTM model evaluated on both the feature-engineered dataset (561 features) and the raw data. The results demonstrate a substantial performance advantage when utilizing the raw data. Across all configurations, models trained on raw data consistently achieved accuracies between 92.54% and 94.24%, significantly outperforming those trained on the manually engineered features, which reached a maximum accuracy of only 86.34% (using SMOTE). This superior performance of the raw data modality is consistently reflected across all other metrics, including the Macro F1 Score and Mean Intersection over Union (MIoU).

An analysis of the raw data configurations reveals that the Baseline (Normal) configuration yielded the highest overall performance, achieving an accuracy of 94.24%, a Macro F1 score of 0.8685, and an MIoU of 0.7929. The superior performance of the unweighted, non-augmented baseline indicates that the CNN-LSTM architecture intrinsically extracts robust spatio-temporal features from the raw data, effectively mitigating the effects of class imbalance. Consequently, the application of external balancing techniques, such as class weighting or SMOTE, was unnecessary for the raw dataset and resulted in a slight degradation of model efficacy.

Upon analyzing the misclassifications, we observed that this framework shares a specific weakness with previous models like SVM and logistic regression: an inability to reliably distinguish between 'Standing' and 'Sitting'. While the individual F1-scores for these classes seem relatively high, isolating them reveals a pairwise misclassification rate of 9.60%. Furthermore, this confusion is highly asymmetrical; the model is over three times more likely to misclassify actual 'Sitting' instances as 'Standing' (14.93% error rate) than the reverse (4.58%). This directional bleed suggests the limitation lies within the overlapping feature representations of the data itself, meaning even a highly complex model will struggle to separate them. To resolve this, expanding the dataset beyond the current 30 subjects would be beneficial. Additionally, incorporating new smartphone sensor features, such as device orientation angles or barometer readings to estimate relative elevation, could provide the distinct signals necessary to differentiate the two postures. Finally, the model's performance on transitional movements seems to have also plateaued, a common issue caused by severe class imbalance. Rather than relying on synthetic sampling methods to address this lack of data, conducting further physical experiments to capture more natural transitions will likely yield better results.

Dataset	Config	Accuracy	Macro F1 Score	MIoU
561 Features	Normal / Baseline	0.8536	0.54	0.4615
561 Features	Class-weight	0.7653	0.62	0.5014
561 Features	SMOTE	0.8634	0.76	0.6310
Raw Data	Normal / Baseline	0.9424	0.8685	0.7929
Raw Data	Class-weight	0.9254	0.8590	0.7726
Raw Data	SMOTE	0.9375	0.8634	0.7789

6. Discussion & Conclusion

Model	Input	Accuracy	Macro F1	MIoU	Params
Logistic Reg.	561 Features	0.926	0.830	-	-
SVM-A	561 Features	0.942	0.860	0.771	-
SVM-B (v2)	561 Features	0.929	0.850	0.758	-
1D-CNN	561 Features	0.840	0.671	0.542	34,764
CNN-LSTM	Raw Sequences	0.942	0.869	0.793	312,012

This project evaluated four model families — logistic regression, kernel SVM, 1D-CNN, and CNN-LSTM — on a 12-class smartphone activity recognition task with severe class imbalance between six well-represented

activities and six rare postural transitions. Three findings emerged.

First, the engineered 561-feature space does most of the heavy lifting for traditional models. Logistic regression reached 93% accuracy and macro F1 of 0.83 (Section 5.1); the best SVM configuration reached 94.2% accuracy and macro F1 of 0.860 (Section 5.2.1). The gap between a linear classifier and an RBF kernel on these features is narrow, confirming that the hand-crafted UCI features already linearise most of the problem and that kernel complexity buys only marginal gains.

Second, class imbalance mitigation is more model-dependent than standard practice suggests. Class reweighting had no measurable effect on the SVM (Section 5.2.1). SMOTE improved SVM-B's macro F1 by three points but could not close the gap between static and transition classes (Section 5.2.2). On the 1D-CNN, class weights degraded macro F1 (Section 4.3). On the CNN-LSTM, both SMOTE and class weighting slightly degraded the raw-data baseline (Section 5.4), because the architecture already learned robust representations from the real minority samples. The common advice to apply SMOTE by default therefore needs qualification: it helps when the underlying representation is static and class-imbalanced, but can harm when the model already learns useful structure from minority data directly.

Third, representation matters more than architecture. The CNN-LSTM trained on the same 561 engineered features achieved macro F1 of only 0.54, well below logistic regression; trained on raw sensor sequences it reached macro F1 of 0.8685 and accuracy of 94.24% (Section 5.4). The same model is either the worst or the best in the study depending entirely on the input. The 1D-CNN on engineered features (macro F1 0.67, Section 5.3) sits between these extremes, outperforming the CNN-LSTM on features but underperforming the SVM, confirming that deep architectures do not automatically compensate for information lost during feature engineering. The SVM's macro F1 plateau at 0.85 across multiple architectures (Section 5.2.2) and the SITTING↔STANDING confusion identified in Section 5.2.1 are both explained by this: aggregate window statistics discard the temporal ordering that distinguishes postural transitions from their reverses.

For deployment, the trade-off is not straightforward. The CNN-LSTM on raw data is the best performer but requires 312,012 parameters and a sliding-window inference pipeline; the SVM on engineered features offers the best accuracy per parameter and is more practical for on-device inference if feature extraction cost is acceptable.

Several limitations bound these conclusions. The dataset is pre-segmented into fixed windows with known activity boundaries, whereas a deployed system must perform online segmentation. Transition classes are sparse even in the full training set, and synthetic oversampling can only interpolate within the small existing support. We evaluated on a single dataset with thirty subjects, so cross-dataset generalisation remains open. Future work should test on continuous streams rather than pre-windowed samples, evaluate compressed CNN-LSTM variants for edge deployment, and adopt a cross-subject evaluation in which entire users are held out to give a more realistic estimate of generalisation performance.

7. Appendix

Declaration of Original Work. By entering our Student ID below, we certify that we completed our assignment independently of all except as stated below.

No Exception to the Class Policy. We followed the CS3244 Class Policy in doing this assignment.

Signed, A0305617H, A0266580X, A0319696J, A0272137H, A0305715J, A0286687A

2. References. We give credit where credit is due. We acknowledge that we used the following websites, AI model, or contacts to complete this assignment:

Anguita, D., Ghio, A., Oneto, L., Parra, X., & Reyes-Ortiz, J. L. (2013). A public domain dataset for human activity recognition using smartphones. In *Proceedings of the 21st European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning* (pp. 437–442).

Arafa, A., Harfoush, H., El-Fishawy, N., & Radad, M. (2026). SMOTE-enhanced CNN-Bi-LSTM for wearable sensor-based human activity recognition. *Pervasive and Mobile Computing*, 117, Article 102161. <https://doi.org/10.1016/j.pmcj.2026.102161>

Cho, H., & Yoon, S. M. (2018). Divide and conquer-based 1D CNN human activity recognition using test data sharpening. *Sensors*, 18(4), 1055. <https://doi.org/10.3390/s18041055>

Dupare, S., Moundekar, Y., & Chandankhede, P. (2023). Implementation of smartphone based activity recognition. In *2023 International Conference for Advancement in Technology (ICONAT)* (pp. 1-5). <https://doi.org/10.1109/ICONAT57137.2023.10080702>

Ignatov, A. (2018). Real-time human activity recognition from accelerometer data using Convolutional Neural Networks. *Applied Soft Computing*, 62, 915–922. <https://doi.org/10.1016/j.asoc.2017.09.027>

Muralidharan, K., Ramesh, A., Rithvik, G., Prem, S., Reghunaath, A. A., & Gopinath, M. P. (2021). 1D convolution approach to human activity recognition using sensor data and comparison with machine learning algorithms. *International Journal of Cognitive Computing in Engineering*, 2, 130–143. <https://doi.org/10.1016/j.ijcce.2021.09.001>

Rabbi, J., Fuad, M. T. H., & Awal, M. A. (2021). *Human activity analysis and recognition from smartphones using machine learning techniques*. arXiv. <https://doi.org/10.48550/arXiv.2103.16490>

Sharma, G., Yadav, T., Yadav, V., & Mahajan, S. (2025). Hybrid CNN-LSTM model for accurate human activity recognition using wearable sensor data. *International Journal for Multidisciplinary Research*, 7(6). <https://doi.org/10.36948/ijfmr.2025.v07i06.62226>

Claude Opus 2.6, which helped format this report, and offered suggestions for performance improvements

Claude Sonnet 4.6, which helped to generate several source code of this report.